

2026年暑期研修

FlagTree

Triton-TLE 语言扩展与编译优化实践

汇报人：肖航

智源研究院 AI 编译器研究员

01 郭晖

TLE: 一种多层次的 Triton 语言扩展

1. 背景
2. TLE 是什么
3. 性能收益
4. Roadmap

02 肖航

Tile IR 后端与编译优化实践

1. FlagTree 集成 TileIR
2. FlagOSTune 优化
3. Convert Layout 优化
4. 指令重排优化
5. fragments合并优化

背景

01

✔ Triton 的核心优势

Triton 是一种 **Python DSL 形式的算子编程语言**，基于 Tile 的编程理念屏蔽了 Thread、存储层级、Layout、流水线、同步等硬件细节，通过编译器优化实现较高性能的算子。

⚙️ 屏蔽硬件细节

⚙️ 自动编译器优化

🔗 Python DSL 语法

👥 庞大社区生态

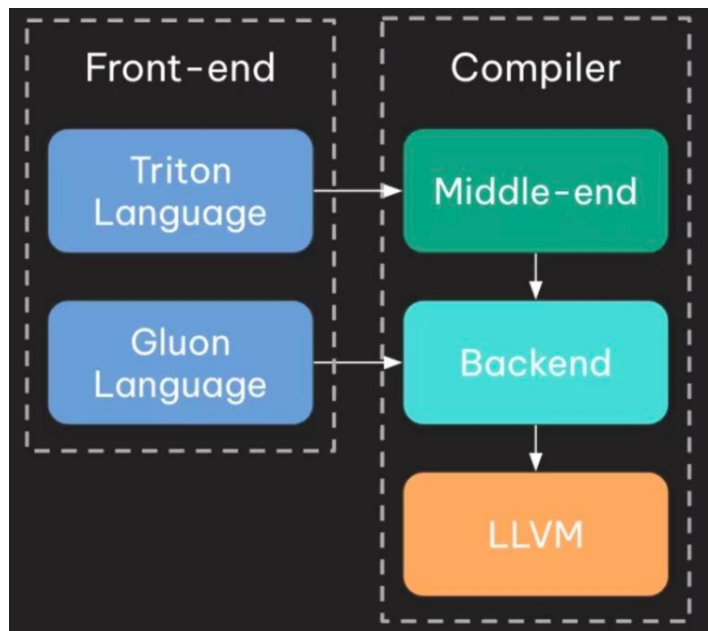
⚠️ 面临的挑战

适配进展缓慢

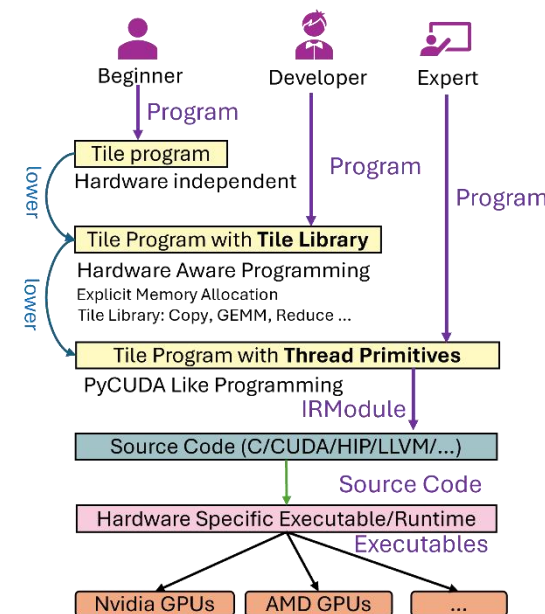
在 DSA 和新 GPU 架构上的适配进展比较缓慢，难以快速响应硬件迭代

难以实现极致性能

在细粒度控制存储层级和并行粒度上缺少抽象，挖掘极致性能难度大



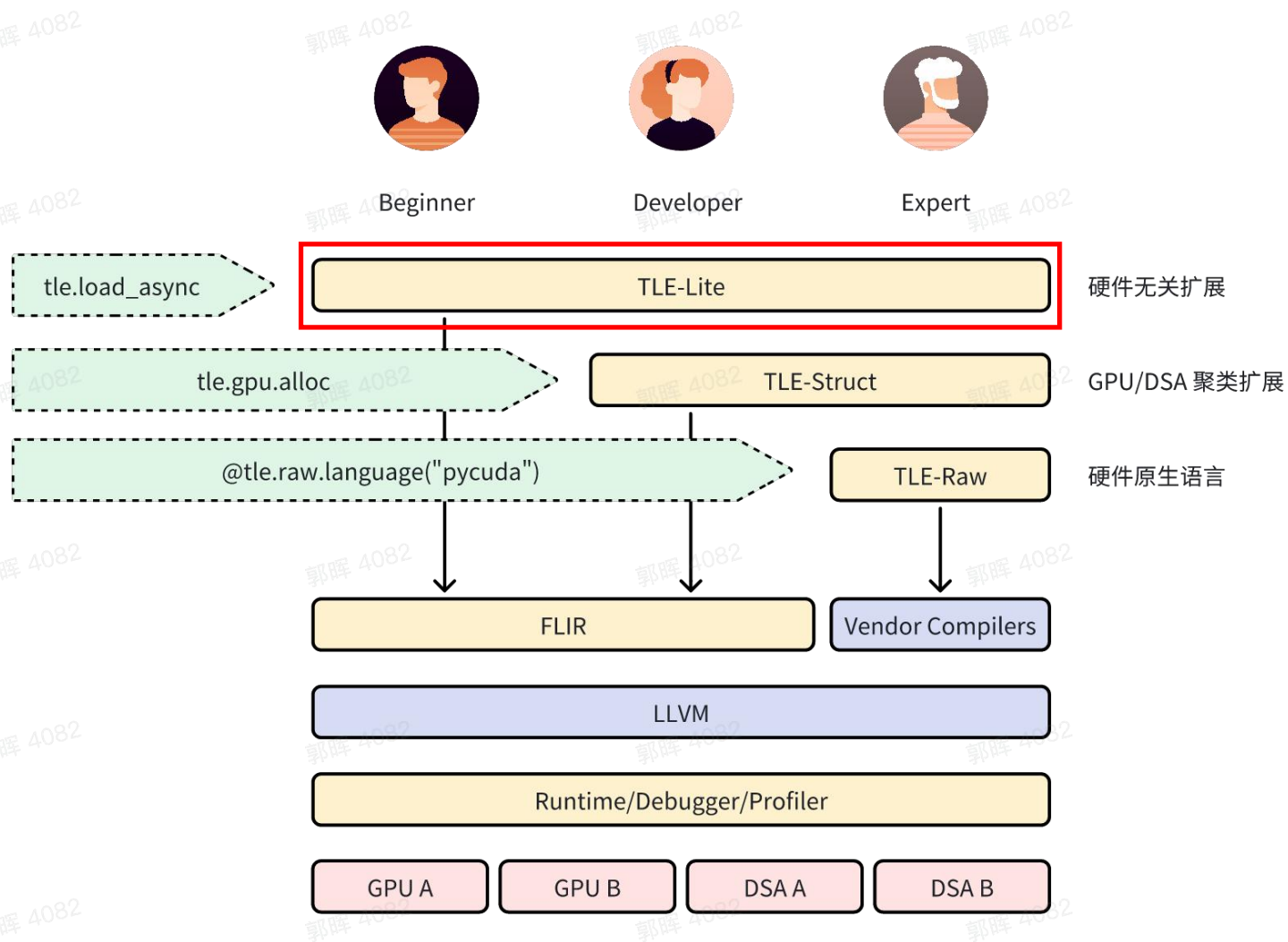
Gluon/TLX



TileLang

TLE 是什么

02



TLE-Lite

轻量级扩展

设计哲学

一次编写，到处运行

核心理念

- 通过引入高层语义提示，引导编译器进行启发式优化
- Triton 表达能力增强

- ✓ 向后兼容性
- ✓ 最小代码侵入性
- ✓ 跨平台性能提升

兼容性

所有特性兼容各类硬件后端

目标用户

算法工程师、快速性能优化场景

✂ `tle.extract_tile`

使用子 tile 的 shape 将输入 tensor 切分为子 tile 网格，并提取指定坐标子 tile。

```
# x is [4, 4], z is [2, 2]
# 将 x 切分为 shape=[2, 2] 的子 tile 网格，取 [0, 0] 子 tile
z = tle.extract_tile(x, index=[0, 0], shape=[2, 2])
```

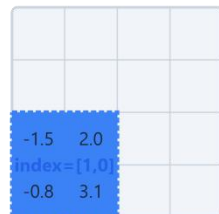
+ `tle.insert_tile`

使用子 tile 的 shape 将输入 tensor 切分为子 tile 网格，并更新指定坐标子 tile。

```
# x is [4, 4], y is [2, 2], z is [4, 4]
# 将 x 切分为 shape=[2, 2] 子 tile，使用 y 更新 [0, 0] 子 tile
z = tle.insert_tile(x, y, index=[0, 0])
```

TLE Tensor 切片与更新流程

1. 原始 Tensor x [4, 4]



```
sub = tle.extract_tile(x, index=[1, 0], shape=[2, 2])
```

Extract

sub [2, 2]

-1.5	2.0
-0.8	3.1

ReLU

Updated sub

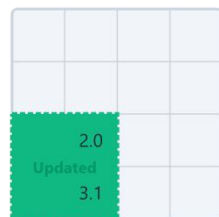
0.0	2.0
0.0	3.1

```
sub = tl.maximum(sub, 0.0)
```

Insert

```
x = tle.insert_tile(x, sub, index=[1, 0])
```

3. 更新后的 Tensor x [4, 4]



典型场景：对子区域做激活

x: [4, 4]

```
sub = tle.extract_tile(x, index=[1, 0], shape=[2, 2]) # rows [2:4], cols [0:2]
```

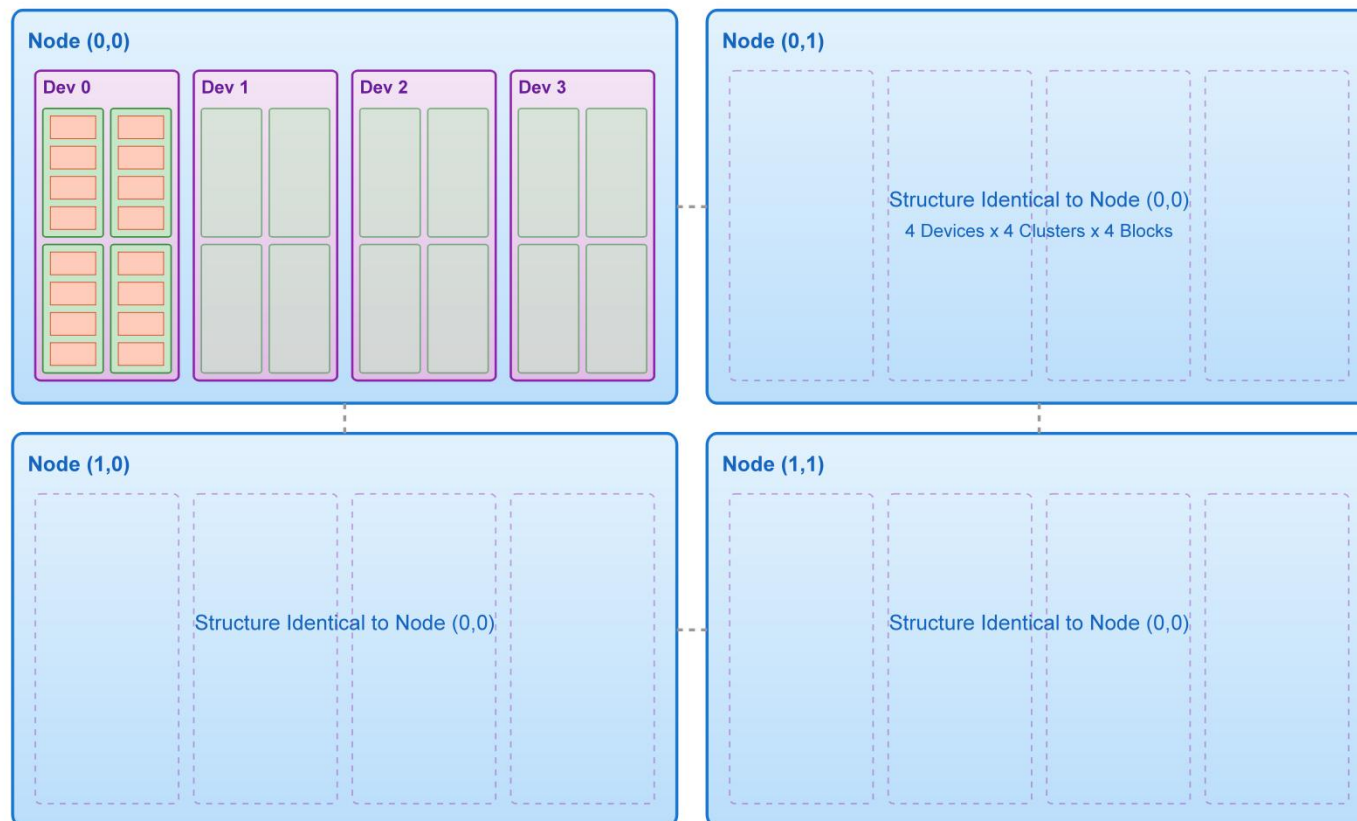
```
sub = tl.maximum(sub, 0.0) # 对子 tile 做 ReLU
```

```
x = tle.insert_tile(x, sub, index=[1, 0])
```

Device Mesh Topology Visualization

Shape: (2, 2, 4, 2, 2, 4) | Total Size: 256

Node (2x2) Device (4) Cluster (2x2)
Block (4)



设备网格

定义逻辑拓扑结构

```
topology = {
    # 跨节点层级 (2x2 = 4 nodes)
    "node": [("node_x", 2), ("node_y", 2)],
    # 节点内 GPU (4 devices)
    "device": 4,
    # GPU 内 Cluster (2x2)
    "block_cluster": [("cluster_x", 2), ("cluster_y", 2)],
    # Cluster 内 Block (4 blocks)
    "block": 4
}

# mesh.shape -> (2, 2, 4, 2, 2, 4)
mesh = tle.device_mesh(topology=topo)
```

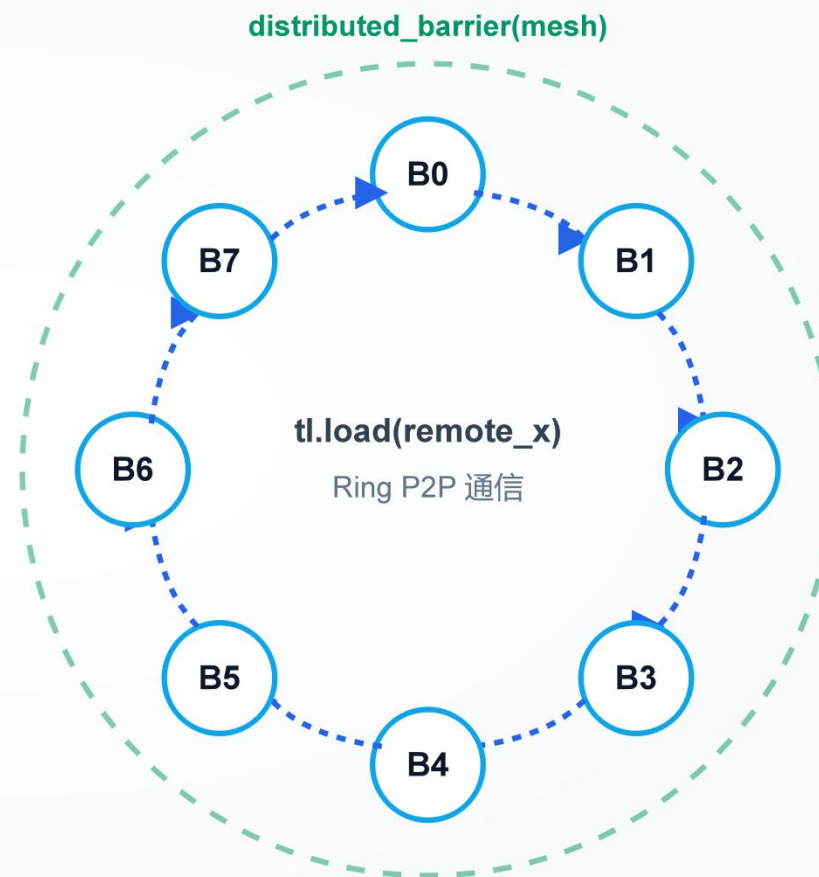
分布式 Device Mesh: Ring 通信与 Barrier 同步



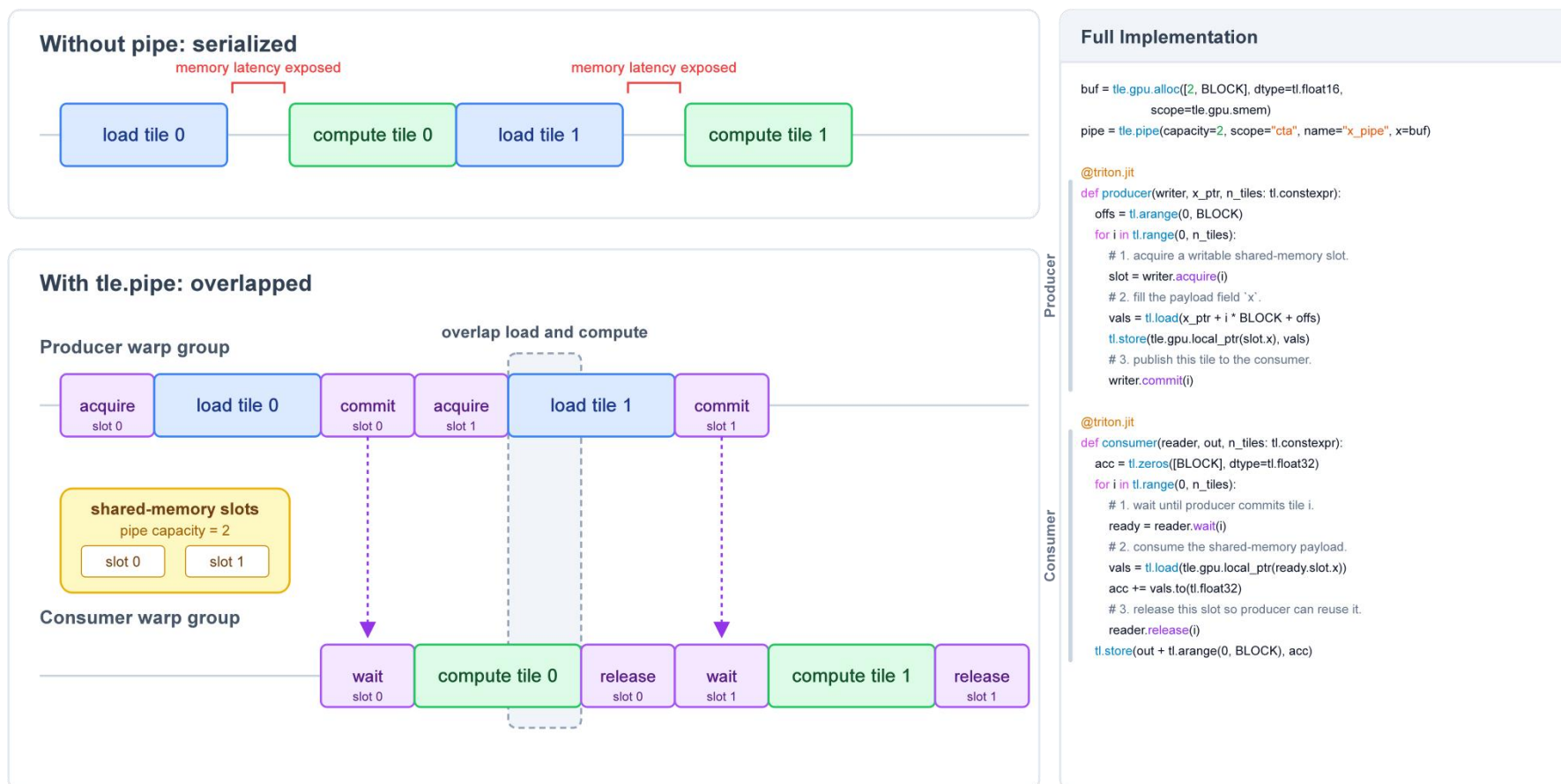
```
mesh = tle.devices.get_mesh({: })
shard_id = tle.shard_id(mesh, )
block_rank = "block"
next_dev_id = (block_rank + 1) % mesh.shape[0]
remote_x = tle.remote(x, shard_id=next_dev_id)
```

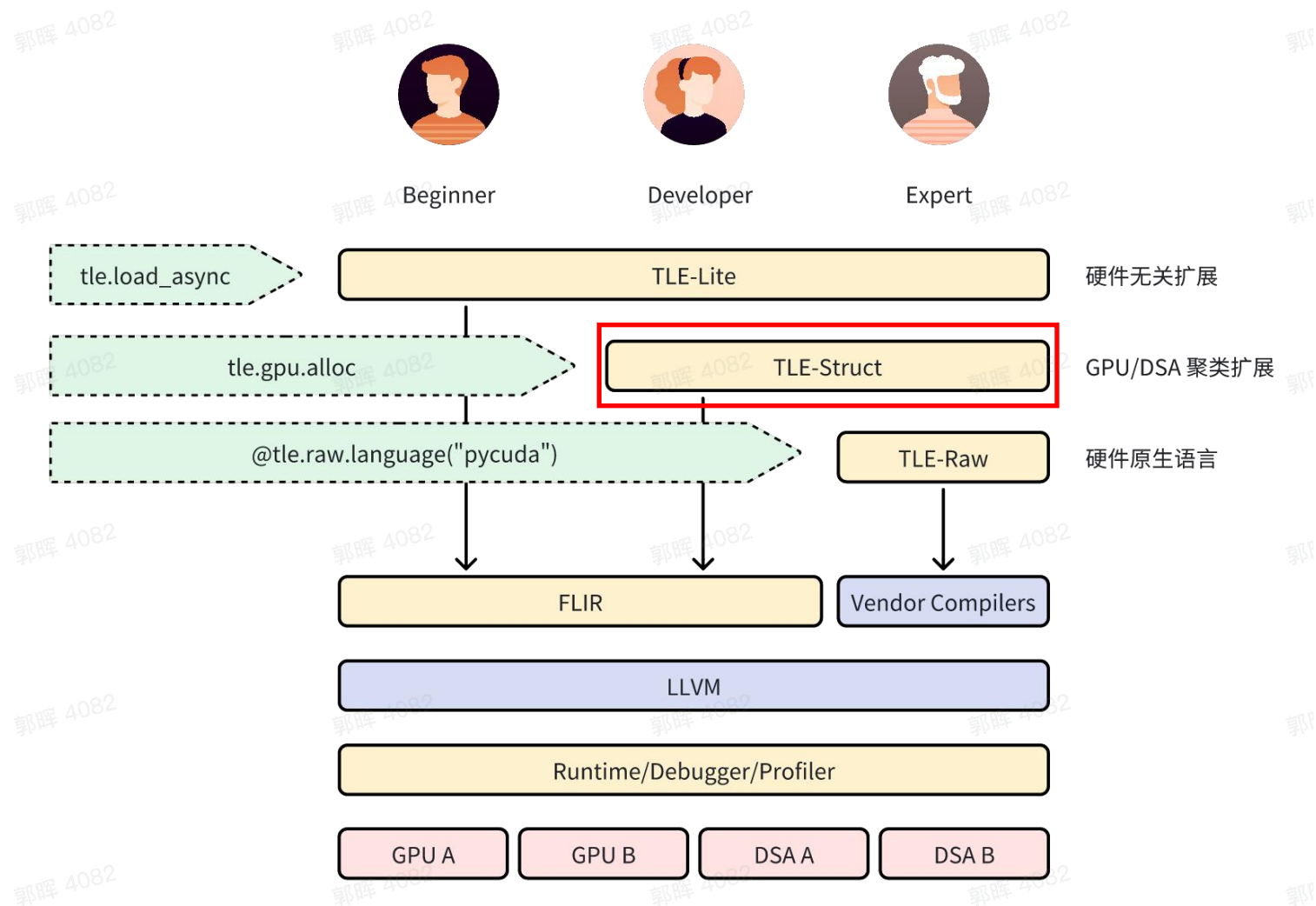
原理解析

1. Mesh 拓扑: 8 个 Block 组成 1D 逻辑网格
2. 远程访问 (P2P): 每个节点计算邻居 Rank 通过 `tle.remote` 获取句柄并 load 数据
3. 全局同步 (Barrier): 通信完成调用 `distributed_barrier` 阻塞等待所有节点



`tle.pipe` 把 CTA 内协作提升为生产者-消费者编程范式：producer 写 slot 并 `commit`，consumer `wait` 后读取并 `release`。同步关系由 pipe 生命周期表达，减少手写 barrier、phase、arrive count 的心智负担，也降低漏等、错发、slot 复用错误的概率。





TLE-Struct

架构感知扩展

设计哲学

架构感知，精细调优

核心理念

依据硬件架构特征将后端划分为GPGPU、DSA等聚类，暴露通用的层次化并行和存储结构

- ✓ 显式定义数据布局
- ✓ 结构化计算映射
- ✓ 发挥硬件差异化能力

硬件分类优化

GPGPU、DSA等聚类提供扩展

目标用户

算子开发工程师

✂ `tle.[gpu/dsa].alloc`

分配内存

```
a_smem = tle.gpu.alloc(  
    [XBLOCK, YBLOCK],  
    dtype=tl.float32,  
    scope=tle.gpu.smem)
```

+ `tle.[gpu/dsa].local_ptr`

获取指定索引对应的内存指针

```
# 取 a_smem[0, :] 的 pointers: [(0, 0), (0, 1), ..., (0, YBLOCK-1)]  
a_smem_ptrs = tle.gpu.local_ptr(  
    a_smem,  
    indices=(tl.zeros([YBLOCK]), tl.arange(0, YBLOCK)))
```

内存管理原语：基于 local_ptr 的直方图统计 (Expert Counting)



```
# 1. block 内统计
= tle.gpu.alloc([BLOCK_EXPERT],
local_counts
dtype=tl.int32, scope=tle.gpu.smem)
base (...):
forinrange
    expert_id = tl.load(topk_ids_ptr + offs, mask=mask, ...)
    = tle.gpu.local_ptr(local_counts, (expert_id, ))
    count_ptrs
    (count_ptrs, , mask=mask,
    tl.atomic_addl
    sem=, scope=)
    "relaxed""cta"

# 2. cluster 合并
```

核心原语解析

tle.gpu.alloc & local_ptr:

在 SMEM 分配 Buffer, 并通过 local_ptr 构建基于 expert_id 的动态指针视图 (Gather View), 实现高效的 CTA 内直方图统计。

tle.remote & atomic_add (Cluster):

获取 Rank 0 的远端指针视图, 利用硬件 Cluster 级原子加法, 将各个 Block 的局部统计结果一键合并至全局, 避免繁琐的规约。

阶段 1: Block 内统计 (CTA Scope)

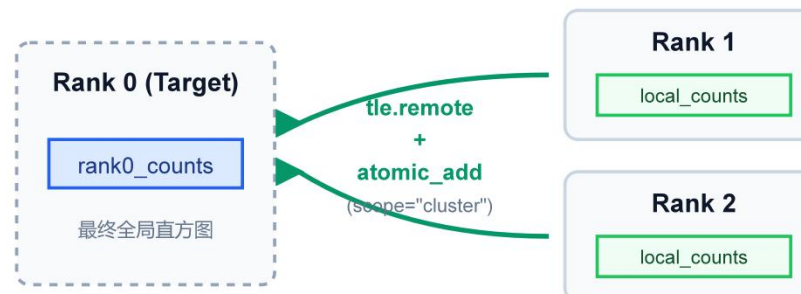
Threads 加载 expert_id:

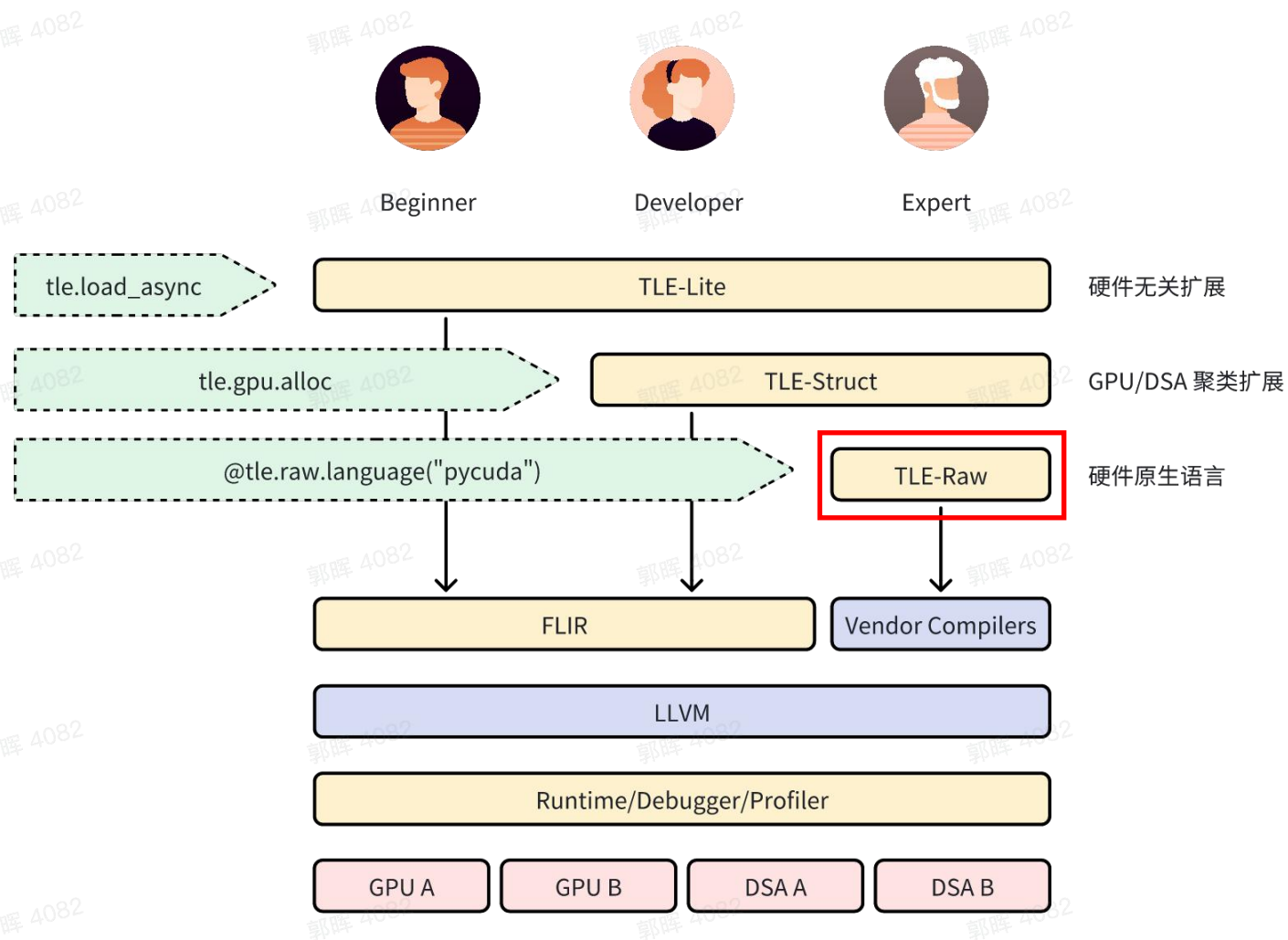


local_ptr(local_counts, (expert_id,))



阶段 2: Cluster 级合并 (Remote Scope)





TLE-Raw

原生控制

设计哲学

原生透传，极致掌控

核心理念

打破 DSL 的抽象边界，支持内联厂商原生代码

- ✓ 厂商原生编译管线
- ✓ 直接生成目标指令
- ✓ 绝对控制权

性能

使用原生编程语言获取最极致性能

目标用户

性能优化专家

Triton & CUDA 混合编程: All-Gather GEMM

1. CUDA C++ (ag-gemm-device.cu)

```
extern "C" __device__ void  
ag_wait_ready(uint64_t *ready, int idx) {  
    nvshmem_signal_wait_until(...);  
}
```

注册

2. Python Dialect 绑定 (ag-gemm.py)

```
@dialect(name="cuda", file="...cu",  
extern_func_name="ag_wait_ready")  
def wait_ready(*args, **kwargs): ...
```

注入到 Triton JIT

3. Triton Kernel 调用 (ag_gemm_consumer)

```
@triton.jit  
def ag_gemm_consumer(...):  
    # 阻塞等待 P2P 通信完成  
    tle_raw.call(wait_ready, [ready, src_rank])  
    acc = tl.dot(a, b.T, acc)
```

Overlap 流水线机制

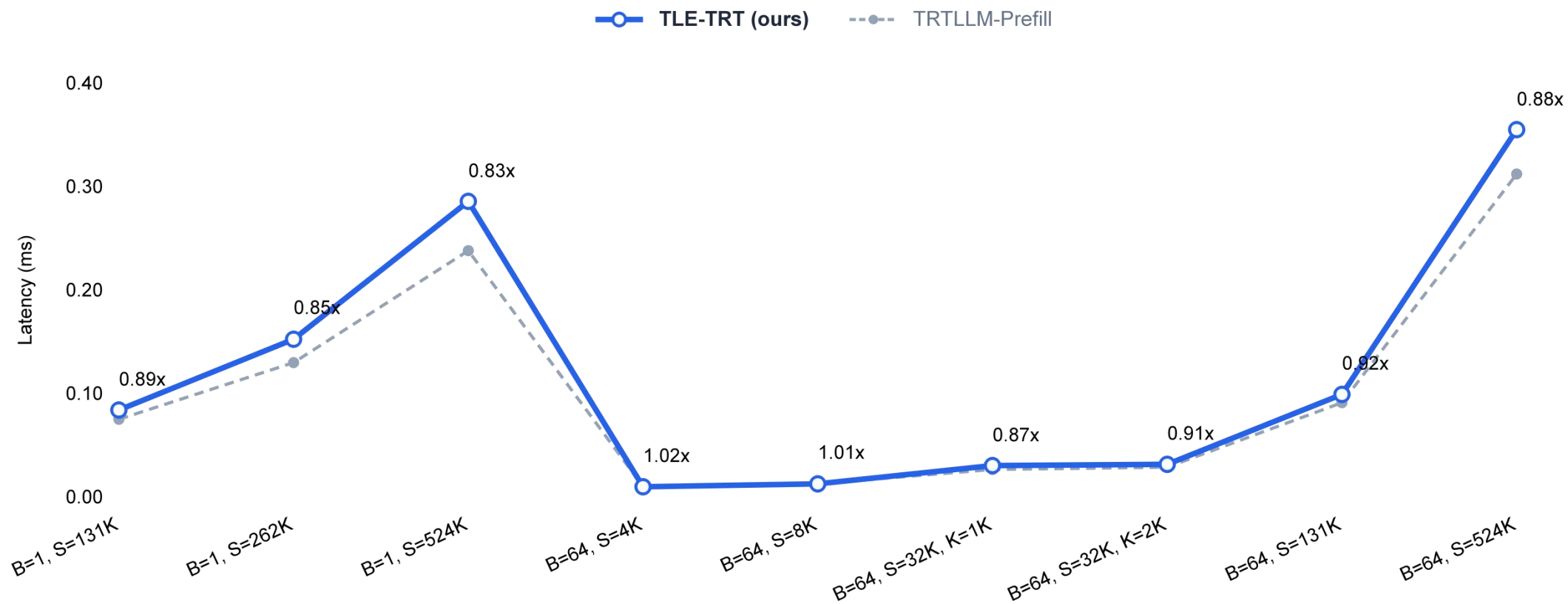
Comm Stream: P2P Copy

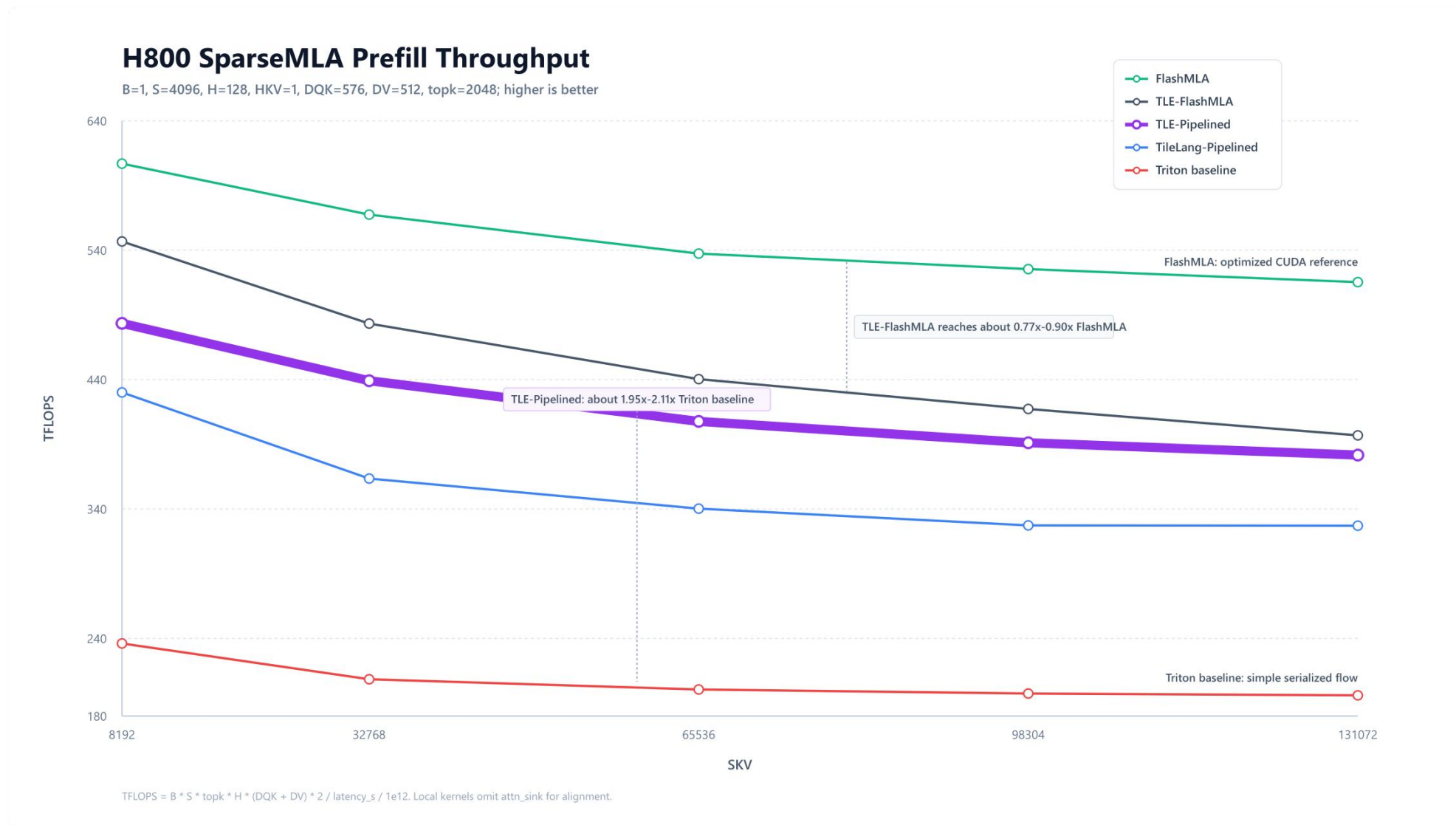
Comm Stream: Set Signal

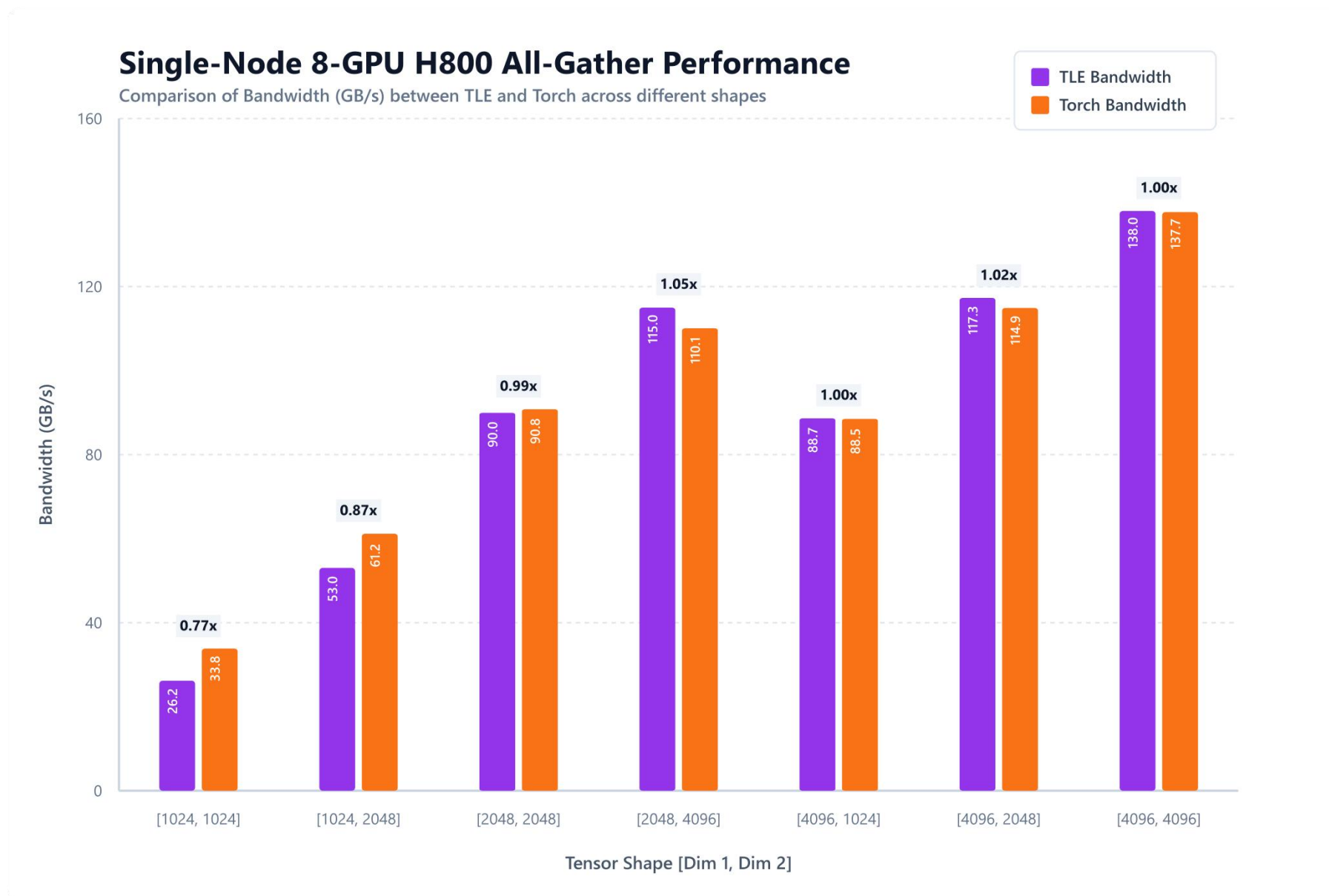
Compute Stream: GEMM (等待信号后执行)

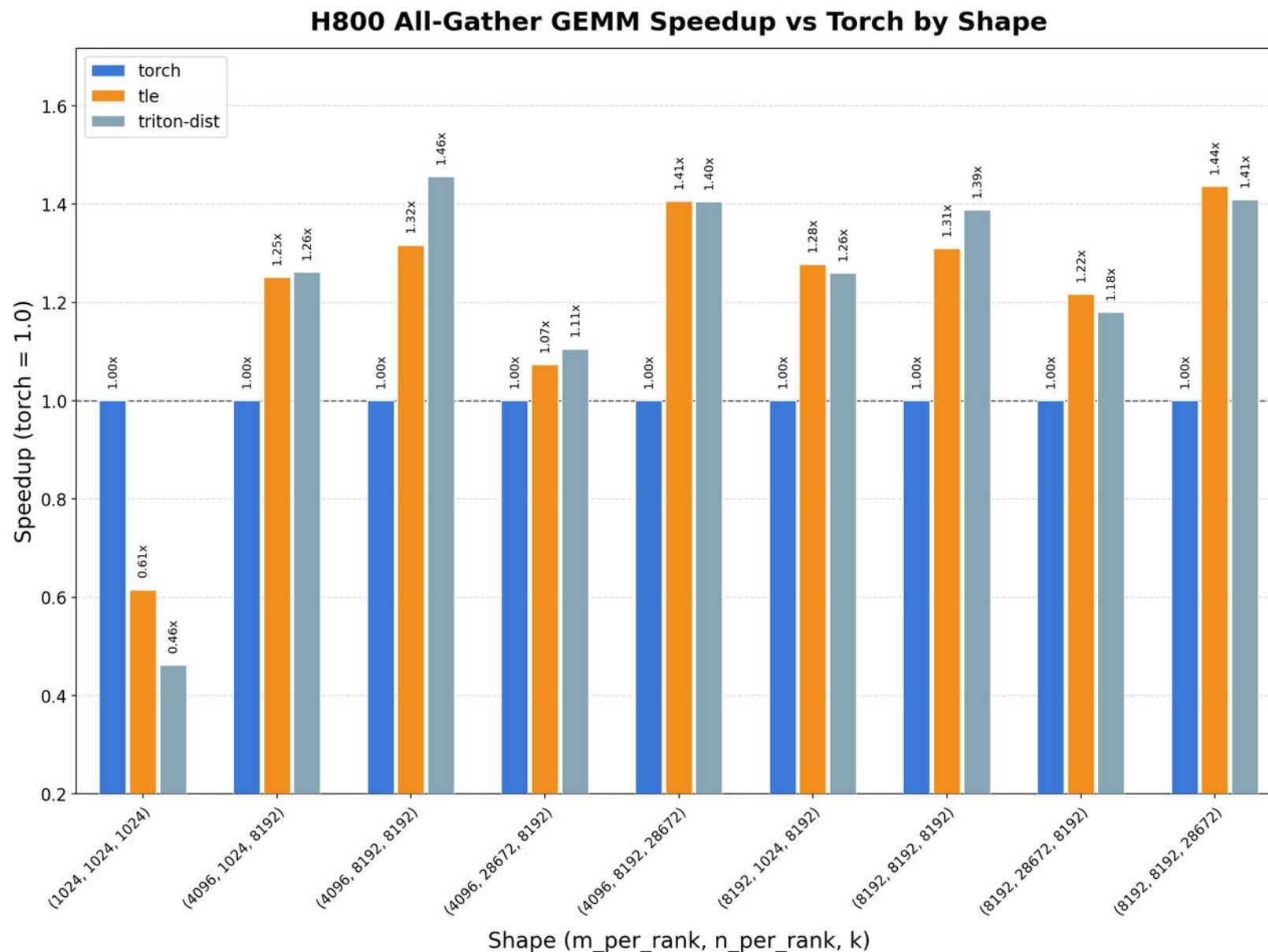
性能收益

03

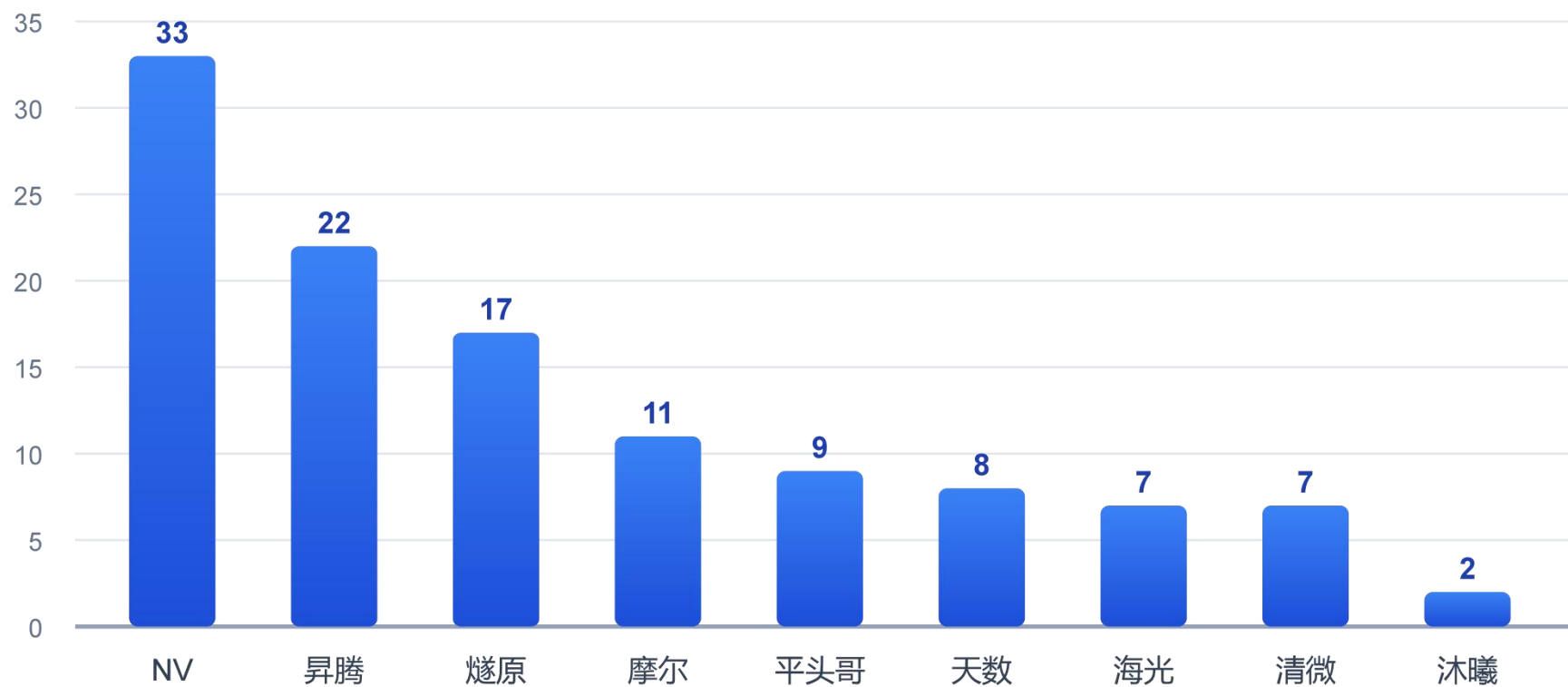








TLE 原语各硬件厂商实现数量



Roadmap

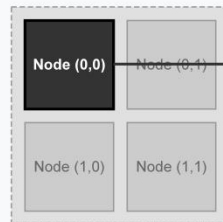
03

Hardware Topology & Logical Tensor Mapping

Mapping: Logical Tensor (Split) -> Device (Axis 1) -> Cluster (Axis 0) -> Block/Psum

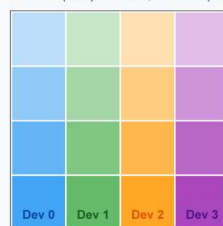
■ Node ■ Device ■ Cluster Splits □ Block ■ Psum

1. Global Nodes (2x2)



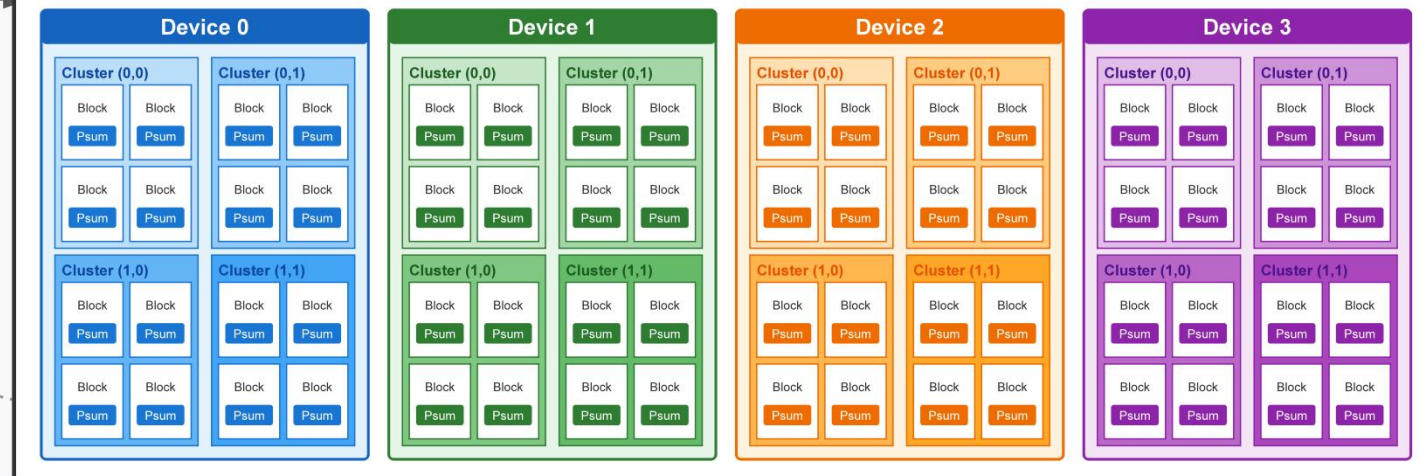
2. Logical Tensor View

Axis 1 (Col) -> Dev, Axis 0 (Row) -> Cluster

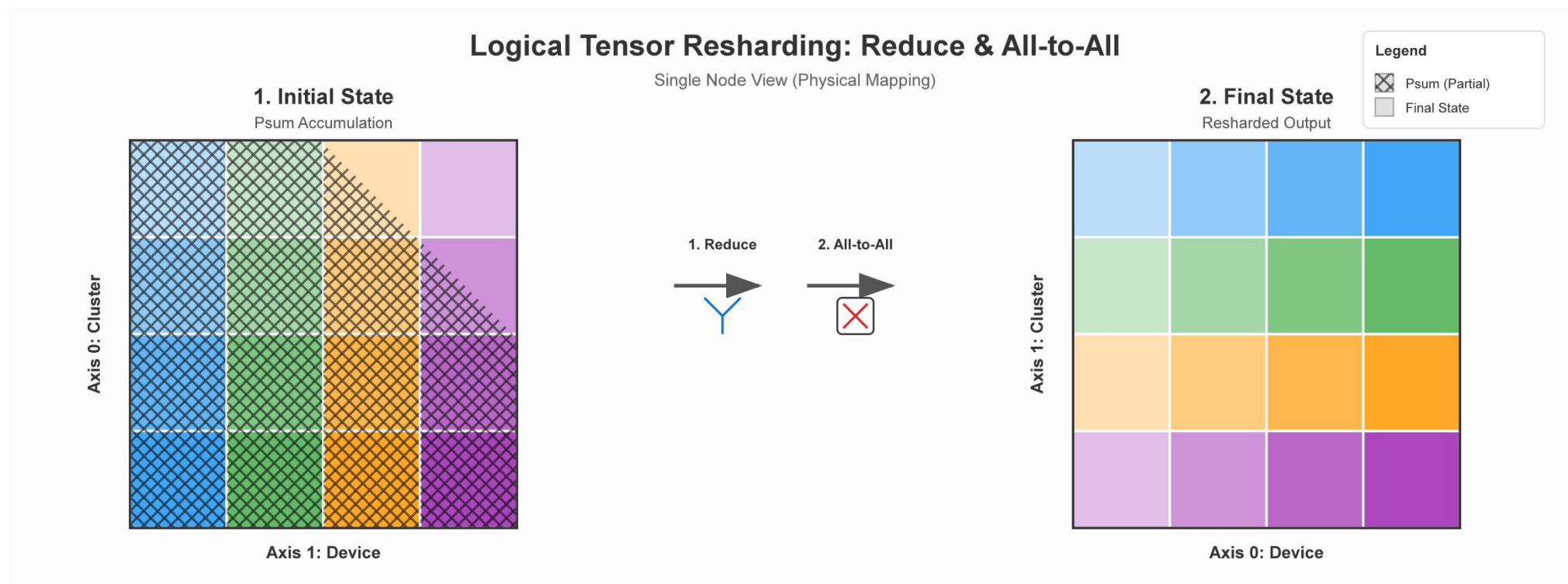


3. Node (0,0) Internal Structure

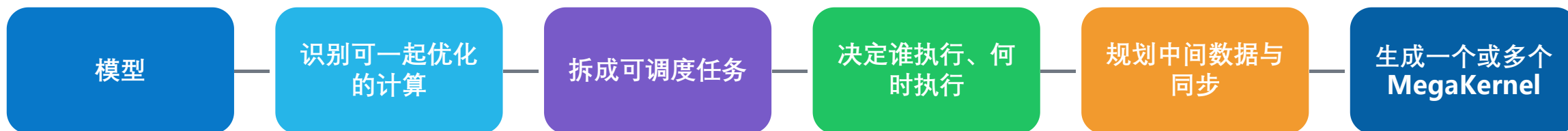
Physical Layout (Colors match Logical Tensor)



```
x_shard = tle.sharding(mesh, split=[["cluster_x", "cluster_y"], "device"], partial=["block"])
x = tle.make_sharded_tensor_descriptor(x_ptr, sharding=x_shard, shape=[4, 4])
```



```
y_shard = tle.sharding(mesh, split=[["device", ["cluster_x", "cluster_y"]]])
y = tle.reshard(x, sharding=y_shard)
```



1 任务

GPU 上具体要完成哪些工作

2 执行计划

这些工作如何排序、并行和流水

3 设备内执行器

GPU 如何读取并执行这份计划

输入模型；新增复杂度由编译器内部承担。

01 郭晖

TLE: 一种多层次的 Triton 语言扩展

1. 背景
2. TLE 是什么
3. 性能收益
4. Roadmap

02 肖航

Tile IR 后端与编译优化实践

1. FlagTree 集成 TileIR
2. FlagOSTune 优化
3. Convert Layout 优化
4. 指令重排优化
5. fragments合并优化

CUDA Tile IR: 让程序表达 Tile, 而非线程映射

Cutile-Python
Tile Program + 3D Tile Grid

CUDA Tile MLIR Dialect

Tile Compute

`tile<MxNxdtype>`

静态 shape · 统一元素类型

`mmaf / mmai`

3D Tile Grid



每格启动
Tile Block
线程映射由后端决定

Views

TensorView

global pointer + shape + strides

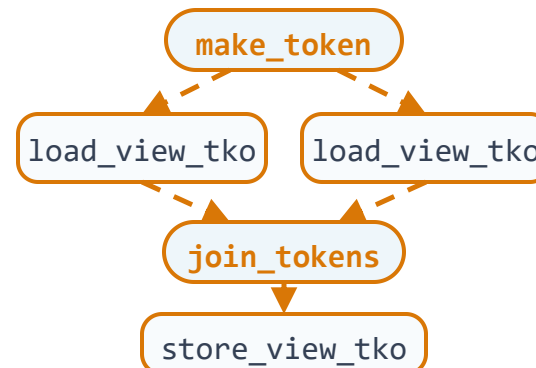
`make_tensor_view`

PartitionView

tile_shape + dim_map
+ OOB padding

`load / store tile`

Token-Ordered Ops



基于Token表达依赖关系, 约束编译器指令执行顺序

→ View data

→ Token dependency

Memory Model

Semantics

weak

relaxed

acquire

release

Scope

tl_blk

device

sys

semantics / scope: 决定同步关系
token: 控制tile block内执行顺序

CODEGEN

Verifier +
Optimizer

Bytecode
Writer

Versioned CUDA Tile IR
Bytecode

CUDA Driver JIT

tileiras AOT

Native Code
cubin

NVIDIA GPU

线程 · 内存 · Tensor Core

程序保留 Tile 级计算、数据视图与必要依赖; 后端决定具体线程映射与硬件实现。

FlagTree × TileIR: 一套 Triton前端, 两条编译路径

@triton.jit Kernel `tl.* + optional tle.*`

FlagTree 公共 JIT / Backend 层

后端扩展点

Backend entry-point discovery

`route_target(target, jit_fn)`

`get_backend / get_driver`

`get_language_extension / make_ir`

Kernel Router

TLE AST classify

`FLAGTREE_USE_TILEIR=1`

TLE Extension API

`tle.* → tl.ext.* → extend_core.py`
`CudaTileSemantic → triton_tileir.cc`

VIEW

`tle.make_tensor_view` `tle.make_view`
`tle.make_partition_view` `tle.dim`

TOKEN / TKO

`tle.create_mem_token` `tle.load_view_tko`
`tle.join_mem_tokens` `tle.store_view_tko`

TileIR Backend

third_party/tileir

每个 JIT Kernel 独立选择 `target=tileir`

TileIRBackend

`TileIRCodeGenerator + CudaTileSemantic`

`TTIR → CUDA Tile IR + view / token ops`
`TritonToCudaTile + AutoGenMemoryToken`

`Bytecode → tileiras → cubin / TileIRDriver`

Native NVIDIA Backend

`target=cuda`

CUDABackend

`TTIR → TTGIR → LLVM / PTX`

`ptxas → cubin`

`CudaDriver`

CUDA Driver API → NVIDIA GPU

全局 `CudaDriver` 管理 device / stream; `TileIRDriver` 负责对应 cubin 的加载与启动

构建 `FLAGTREE_BACKEND=tileir python3 -m pip install . --no-build-isolation`

运行 `FLAGTREE_USE_TILEIR=1 python your_kernel.py`

FlagOSTune: Triton Autotune 又快又准

问题一 | 默认 Shape 与 Config 覆盖有限

121 → 1,994 真实 MM Shape

6 个模型 × 4 类场景; 216 个默认配置覆盖不足。

16.48×

问题二 | 扩大搜索空间后调优成本不可接受

3,360 Config / Shape

结合 1,994 个真实 Shape, 理论上限约 670 万组 Shape-Config 组合。

15.56×

1 搜索空间扩展

模型真实 Shape

M / N / K · stride · dtype



Expand Config / 参数空间

BLOCK_M / BLOCK_N / BLOCK_K
GROUP_M · warps · stages



Operator Adapter

特征构造 · 合法性约束
op_id / variants / params / features

2 模型辅助搜索

XGBoost 全候选排序

输出 Top-K (默认 10)

模型预测

全量 YAML 参数空间 + 部分 shape 训练



Top-K 真实编译 / GPU Benchmark

形成实测种子 · LibTuner 回调



GA 遗传搜索

精英保留 · 交叉 · 变异 · 随机探索
少量新候选再实测

闭环

3 实测选优与复用

Best Config

从已实测候选中选优



FlagGems LibTuner 执行

Top-K 实测 + GA 新候选



缓存复用 · SQLite

ConfigCache

BenchmarkCache

命中直接复用

FlagOSTune: 跨平台性能收益 + 调优效率

NVIDIA H20 · 11 Ops

1.21–3.78×

W8A8 FP8 Matmul 3.78×

BMM 3.06× · BaddBMM 2.87×

Moore Threads · 5 Ops

2.58–7.35×

MM 7.35x

BMM 2.58x

AddMM 5.92×

BaddMM 5.25×

MetaX · Mul

1.55× AVG

0.964–2.115×

跨平台复用扩展空间

调优效率: 8×H20 · BF16 · MM· 202 shapes

每个 shape 3,360 个配置; Expanded 作为性能参照

新测试候选总数



622,566 · Expanded

4,070 · FlagOSTune

–99.35% · 约 1 / 153

调优时间

120×

相对性能

99%

Convert Layout: RLC 四阶段增强消除转换开销

batch_norm 简化 TTGIR (基线)

```
%mean = tt.reduce %input : #blocked → #blocked1
%0 = ttg.convert_layout %mean // cvt 1
%bcast = tt.broadcast %0
%normed = arith.subf %input, %bcast
%var = tt.reduce ... : #blocked1
%1 = ttg.convert_layout %var // cvt 2
%2 = ttg.convert_layout %result // cvt 3
tt.store %out_ptr, %2 : #blocked
```

一次 cvt 的路径与开销



成本模型: 对访存密集型算子, 少数几条 cvt 就足以成为 kernel 热路径瓶颈

elements = 128 * 128 # 16384

bytes = elements * 4 # 65536

cost = 32 * max(bytes, 128) = 2,097,152

上游布局决策

Coalesce / MMA
Store

cvt 残留

shared-memory
round-trip

RLC Enhanced · RemoveLayoutConversions

Phase 1a

全局 Cost-based
冲突选择

Phase 1b

Store / Atomic
反向布局传播

Phase 2

小连通分量
proposal + veto

Phase 3

Store write-back
rematerialization

原生 RLC 主要从 anchor 做前向传播, 并用固定规则解决冲突: memory op 优先 blocked, 其它场景偏 MMA。该策略缺少下游 store/atomic 的反向传播, 也没有全局 cost 视角, 因此 RNG、mask、reduction、MMA-result write-back 等链路仍容易残留 cvt

统一候选 layout → 删除冗余 cvt → 降低带宽与 epilogue 开销

cvt elimination 68%–79%

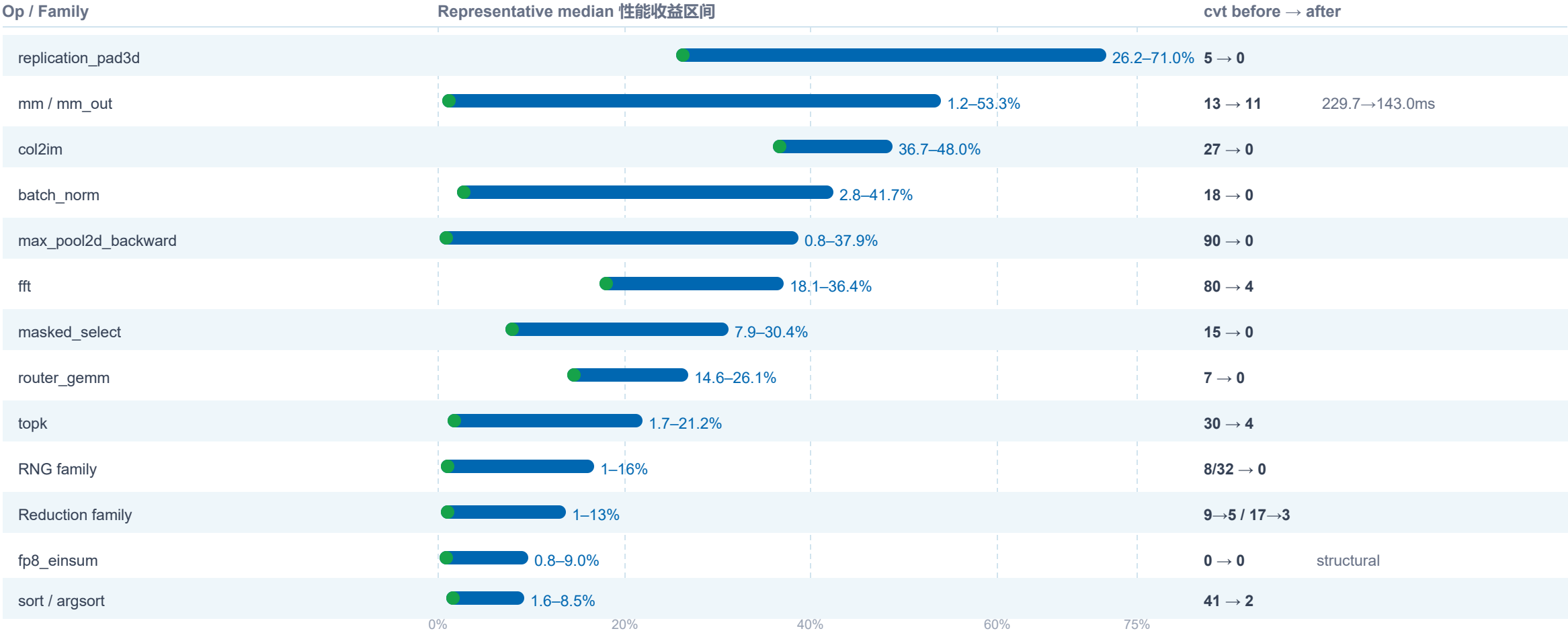
Convert Layout优化性能收益

访存密集、小算子与 MMA write-back 均受益

120+ 受益Ops
0.5%~71%

68–79% net cvt
消除率

最高单类区间
replication_pad3d +26.2%–71.0% · cvt 5→0



指令重排：循环展开后的 Load Fronting

在寄存器路径提前发射多组 load，隐藏 global memory latency

per_token_group_fp8_quant 核心循环

```

for i in tl.range(0, groups_per_program,
    loop_unroll_factor=4, reorder=True):
    y = tl.load(y_ptr + i * group_size + cols)
    max_g = tl.max(tl.abs(y), axis=0)
    scale = max_g / fp8_max
    y_q = tl.clamp(y / scale, fp8_min, fp8_max)
    tl.store(y_q_ptr + i * group_size + cols, y_q)
    tl.store(scale_ptr + i, scale)

```

API → IR 属性 → 执行序列

```

scf.for ... { tt.loop_unroll_factor = 4
tt.reorder = true }

```

重排前

L0 C0 Q0 S0

L1 C1 Q1 S1

L2 C2 Q2 S2

L3 C3 Q3 S3

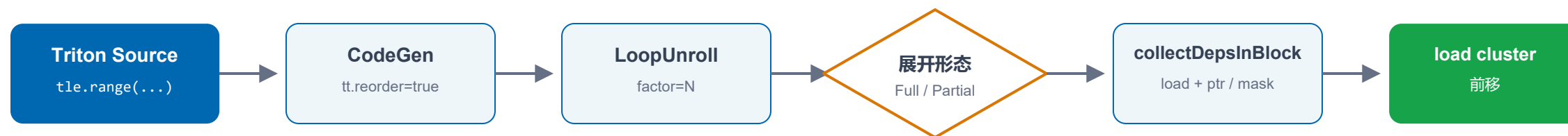
重排后

L0 L1 L2 L3

C0S0 C1S1 C2S2 C3S3

compute / quant / store 的相对顺序不变；只前置安全的 load 与地址依赖。

编译器实现：合入 LoopUnroll，覆盖 Full / Partial unroll



指令重排优势

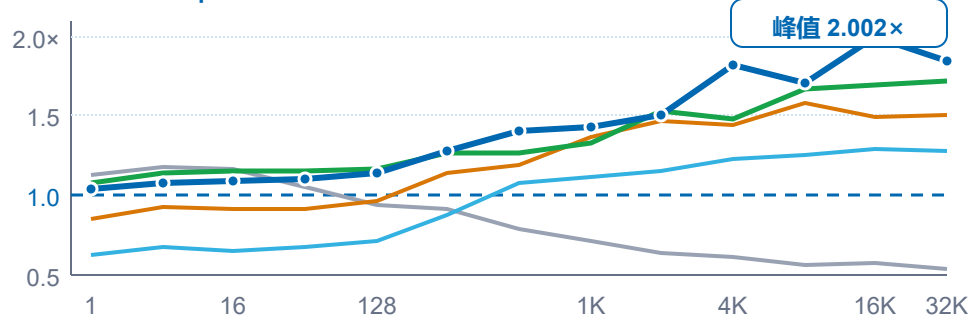
单纯的循环展开只能减少 CTA 数，不能保证 load 前置；num_stages 软流水可用 async copy、shared memory 和 barrier 隐藏延迟，但会引入 `LDGSTS + LDS + BAR.SYNC` 等额外开销。指令重排的目标是在寄存器路径上把多组 load 提前发射，避免额外 shared-memory staging

指令重排优化性能收益

per_token_group_fp8_quant

K=4096 · group=128 · speedup = Torch latency / Gems latency

— base — static — pipeline — unroll+compiler reorder
— unroll+compiler reorder

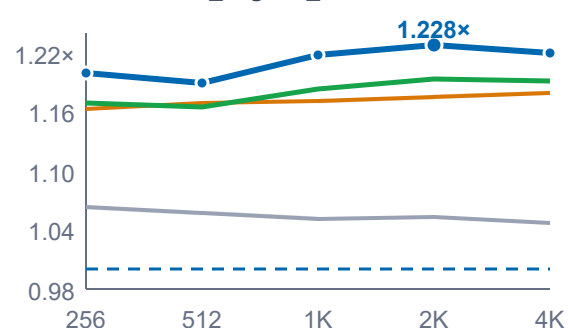


13 shapes · compiler reorder 1.040–2.002× · 平均 1.420×

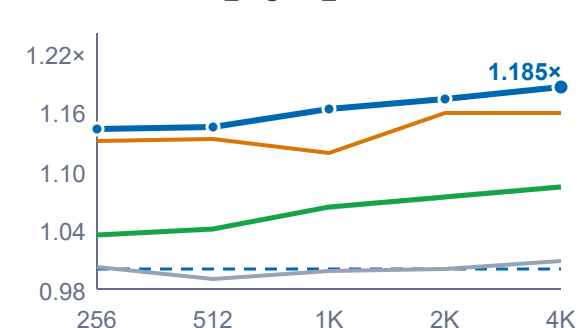
fused_inv_rope_fp8_quant

— origin — pipeline — unroll+reorder — unroll + compiler reorder

tma_aligned_scale = False



tma_aligned_scale = True

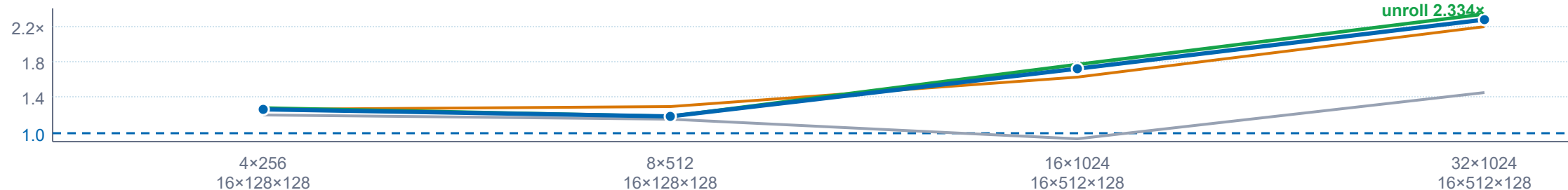


10 shapes · compiler reorder 1.142–1.228× · 平均 1.186×

F/T 峰值 1.228× / 1.185×

cp_gather_indexer_k_quant_cache

— origin — pipeline — unroll — unroll+compiler reorder



4 shapes · compiler reorder 1.182–2.270×

平均 1.609×

重排适合多条独立 load (不能向量化)、计算较轻的量化/缓存 kernel; 相比流水避免 shared-memory staging 与 barrier。

fragments 合并优化，提高内存访问效率

Fused_marlin_moe FP4 | 8× Activation Load + 8× 高层 Dot → 1× Load + 1× 高层 Dot

01 | 问题代码：K=128 被拆成 8 段

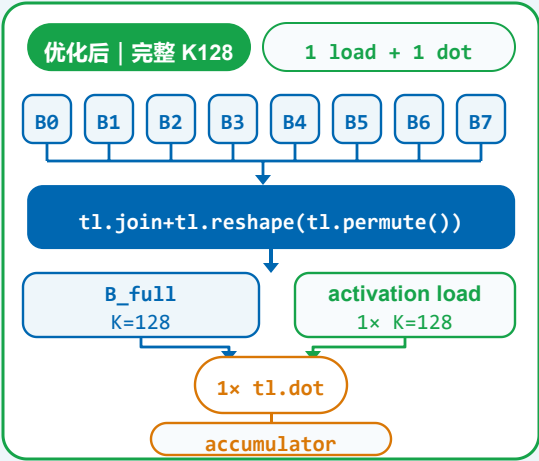
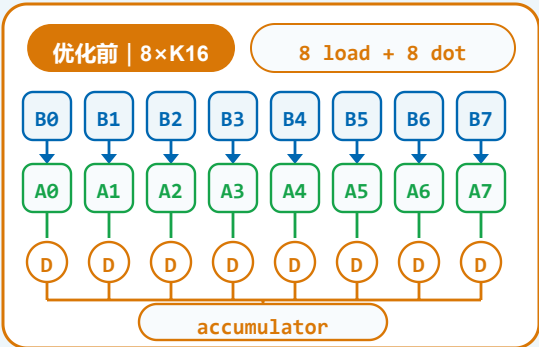
```
bs = dequant(b_packed, scales)
# 8 × [N, K=16] dot fragments
for j in tl.static_range(8):
    a_j = tl.load(a_ptrs + j * K_PACK
                  * stride_ak)
    acc = tl.dot(a_j, bs[j], acc=acc)
```

优化前 TTGIR | 8 段 Dot 串行累加

```
%bs:8 = tt.elementwise_inline_asm "...
%acc0 = ttng.warp_group_dot
    %bs#0, %a0, %cst
%acc1 = ttng.warp_group_dot
    %bs#1, %a1, %acc0
...
%acc7 = ttng.warp_group_dot %bs#7, %a7, %acc6
```

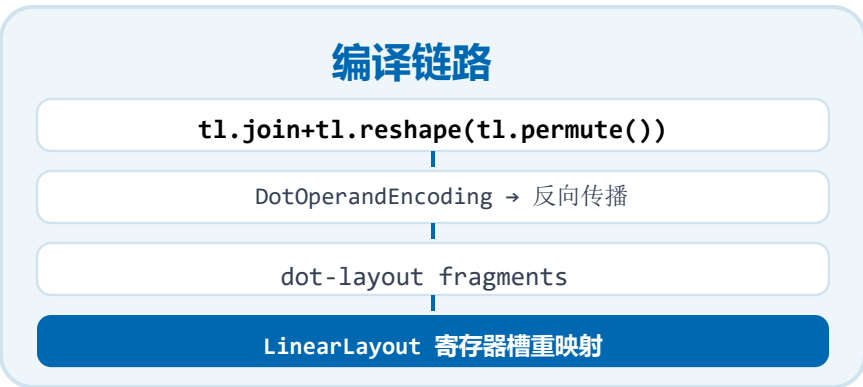
每个 K tile: 8 次地址计算、load 与高层 dot

02 | 优化方案：前后对照



03 | 优化后代码：完整 K 一次供数

```
bs = dequant(b_packed, scales)
# 合并为1个fragment
j0 = tl.join(bs[0], bs[1])
p0 = tl.join(j0, j1)
q = tl.join(p0, p1)
bs_full = tl.reshape(tl.permute(q, (4, 3, 2, 0, 1)))
a_full = tl.load(a_full_ptrs)
acc = tl.dot(a_full, bs_full, acc=acc)
```



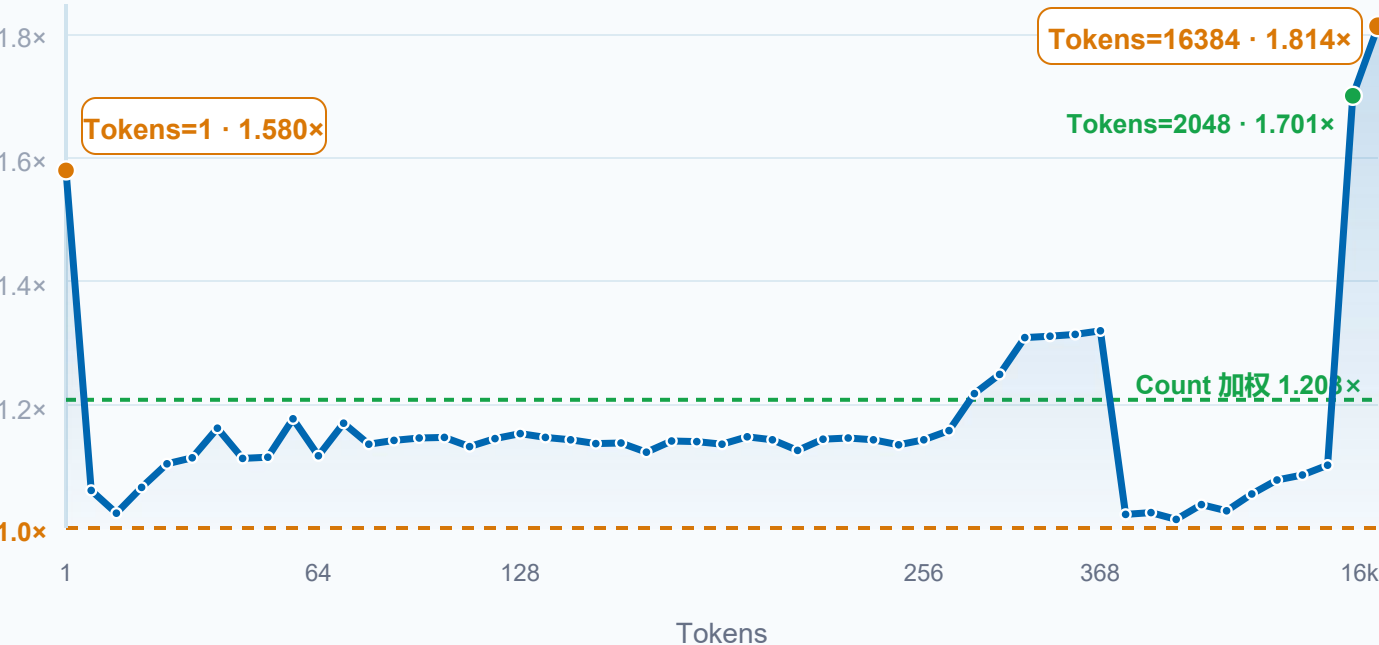
算法优化	01 Padding-aware m-policy	02 Singleton / small_grouped 对齐	03 FlagOSTune
	04 融合 GEMM1 + SiLU	05 BF16 E8M0 scale folding	06 E2M1 位解码优化

Fused Marlin MoE FP4 : 性能收益

算子 Speedup = vLLM CUDA latency / FlagGems latency

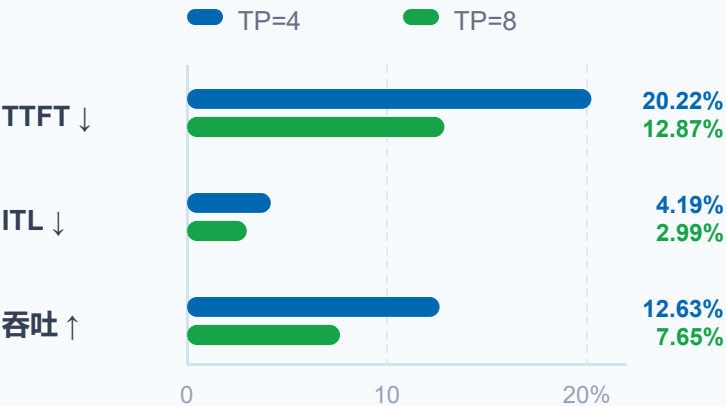
DeepSeek-V4-Flash P32kD1K Shape

1.014–1.814x, avg 1.168x



DeepSeek V4 端到端收益

相对 vLLM CUDA 的改善幅度 (%)



TP=4 | TTFT -20.22% · 吞吐 +12.63%

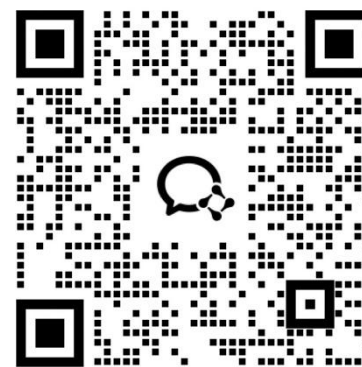
TTFT 11803→9417 ms · ITL 15.50→14.85 ms
总吞吐 4928→5550 tok/s

TP=8 | TTFT -12.87% · 吞吐 +7.65%

TTFT 8534→7436 ms · ITL 13.03→12.64 ms
总吞吐 6220→6696 tok/s



<https://github.com/flagos-ai/FlagTree>



FlagOS 技术交流群

扫描二维码加入群聊